Sistema HCD IA

Informe técnico de entrega académica

Autora: Lic. en Química Silvia Andrea Gómez Villagra.

Fecha: 29 de septiembre de 2026.

Proyecto: procesamiento local de historias clínicas digitales de Salud Mental provenientes de HMIS/VADIGU.

Repositorio: https://github.com/silvitagomez572-alt/sistema-hcd-ia

Rama de entrega: hardening-final.

Commit de referencia técnica: 1588703ce6da742842fe6e856918d716aa78f5f0

Este documento describe el código corregido y la evidencia obtenida durante el cierre técnico. Contiene únicamente información técnica y resultados de fixtures sintéticas. No contiene historias clínicas, nombres de pacientes, documentos personales, bases reales ni secretos.

La entrega es académica y técnica. Aprobar tests de software no demuestra validez clínica ni habilita por sí solo el uso hospitalario. El PDF se incorpora en un commit documental posterior al commit de referencia técnica; el hash exacto del archivo entregado se identifica en el enlace de publicación, sin intentar incluir dentro del PDF el hash de su propio commit.

1. Problema, objetivos y alcance

Las exportaciones hospitalarias combinan texto narrativo, encabezados administrativos y tablas heterogéneas. Su revisión manual dificulta obtener una descripción consistente de la actividad profesional y de las intervenciones registradas. El proyecto busca transformar esas exportaciones en información estructurada, manteniendo el procesamiento local y la trazabilidad de los resultados.

El objetivo del cierre fue corregir fallos técnicos que impedían una ejecución defendible: configuración, entradas, persistencia, privacidad, errores de servicios externos locales y reproducibilidad. Se preservaron las reglas clínicas y el esquema SQLite. El diccionario de profesionales se externalizó con autorización expresa, conservando exactamente sus valores y comportamiento.

Alcance funcional

• Ingesta HCD en HTML, XLS que contiene HTML de VADIGU y TXT. El endpoint de HCD rechaza Excel binario/XLSX en lugar de interpretarlo incorrectamente como HTML.

• Segmentación, conteo y clasificación híbrida por área profesional; interconsultas y métricas estructuradas.

• Censo con lectores CSV, HTML/XLS, XLSX y PDF; internaciones como contexto operacional complementario.

• OCR local con Tesseract y Poppler; el censo PDF escaneado no incorpora OCR automáticamente.

• Recuperación sobre protocolos permitidos, separada de generación Ollama.

• FastAPI, Streamlit, SQLite, pruebas automatizadas y despliegue Docker del núcleo.

Estado de entrega

Componente	Evidencia obtenida hoy	Límite de la conclusión
FastAPI	HTTP real, procesamiento sintético y reinicio	No es prueba de carga ni validación clínica
Streamlit	Salud HTTP real; 13 módulos y API caída mediante mocks	No se ensayó una operación hospitalaria completa
OCR	Tesseract real, idioma español e imagen sintética	No valida documentos escaneados hospitalarios
SQLite	Persistencia tras reinicio e integrity_check	Sin migrar ni probar destructivamente la base existente
Docker	Config, build y up; API/UI y procesamiento sintéticos	Imagen base sin RAG semántico ni modelos privados
RAG	Chroma y embeddings locales reales; benchmark sintético	Corpus pequeño, sin evaluación clínica ni generación
Ollama	Tags y generación breve con modelo ya instalado	Solo disponibilidad; calidad clínica no medida

2. Estado inicial y procedimiento de auditoría

Se trabajó en `/home/silvia/sistema-hcd-ia`, rama `hardening-final`, ya existente al comenzar. HEAD inicial: `4508f53`. Había modificaciones previas en `api/main.py`, `api/tests/test_api.py`, `frontend/app.py` y el módulo de censo. Se guardó una copia privada antes de editar; la procedencia está diferenciada en `entrega/PROVENIENCIA.md`.

La auditoría revisó árbol y archivos versionados, README, dependencias, Docker, tests, API, frontend, pipeline, documentación, scripts e integración continua. La inspección sintáctica encontró 14 archivos Python válidos. No se volcaron filas de la base real ni texto clínico al informe.

Verificación inicial	Resultado
Directorio	/home/silvia/sistema-hcd-ia
Rama	hardening-final, con cambios previos
Python	3.12.3
Docker / Compose	29.1.3 / v2.40.3
Ollama cliente	0.24.0
Recursos observados	7,6 GiB RAM; aproximadamente 292 GiB libres al inicio
Modelo institucional	Artifact local existente; fuera de Git
Fuente de entrenamiento	No hay script institucional versionado para regenerar ese artifact

Baseline y diagnóstico de instalación

El entorno `.venv` previo se preservó. Se creó `.venv-hardening` limpio. Dentro del sandbox fallaban la resolución DNS de PyPI y la conexión HTTPS antes de establecer TLS. Se solicitó acceso de red y la instalación terminó sin desactivar TLS. No se interpretó la ausencia de distribuciones como incompatibilidad mientras el índice era inaccesible.

Baseline previo suplementario: 13 tests aprobados en 24,70 s. Baseline limpio: 13 aprobados en 9,79 s, 0 fallidos y 0 omitidos. Ambos usaron SQLite temporal. TestClient se bloqueaba dentro del sandbox incluso con FastAPI mínimo; la ejecución autorizada fuera del sandbox resolvió ese bloqueo.

El artifact registraba scikit-learn 1.6.1. La instalación sin versiones había resuelto 1.9.1 y produjo advertencias de incompatibilidad. Se fijó 1.6.1; una predicción sintética con el artifact existente terminó sin advertencias de versión. Esto verifica carga e inferencia básica, no su precisión clínica.

3. Arquitectura y flujo de procesamiento

```
Exportación local HCD
    -> límite de entrada y validación de formato
    -> pseudonimización de identificadores detectables
    -> segmentación y reglas de conteo preservadas
    -> clasificación híbrida
    -> métricas y referencias pseudónimas
    -> SQLite local -> API -> interfaz Streamlit

Protocolos del manifiesto -> índice local ChromaDB
    -> recuperación /rag/consulta
    -> generación local opcional /rag/consultar
```

Capa	Responsabilidad	
api/runtime.py	Configuración local, autenticación básica, límite de solicitudes, errores y logging seguro	
api/main.py	Orquestación de endpoints y lógica clínica conservada	
pipeline/	Censo, internaciones, OCR y pseudonimización	
api/rag_service.py	Corpus permitido por hash, embeddings locales y recuperación	
api/llm.py	Transporte Ollama restringido a servidores locales y manejo de fallos	
frontend/app.py	Visualización y solicitudes con timeout, configuración y manejo de caída	
OCR	Tesseract real, idioma español e imagen sintética	No valida
SQLite	Persistencia estructurada; esquema previo conservado	

Las rutas de modelo, base, diccionario, cargas, censo, corpus, embeddings y Ollama se configuran mediante variables de entorno. `.env.example` no contiene secretos. El inicio de FastAPI ya no crea la base como efecto secundario de importar el módulo; la inicialización ocurre en el ciclo de vida del servicio.

El refactor fue incremental: se extrajeron límites HTTP, transporte LLM y servicio RAG. Los monolitos de API y Streamlit siguen siendo deuda técnica. No se realizó una reescritura total ni una migración de esquema.

Contratos relevantes

`/health`, `/docs`, `/hcd/procesar-modelo`, `/hcd/reporte-total`, `/hcd/reportes`, `/censos/upload`, `/mobile` y `/mobile/upload` se mantienen. Se incorporaron `/rag/consulta` para recuperación y `/llm/resumir` para resumen; los endpoints anteriores continúan disponibles según su propósito.

Las entradas tienen límites y validación. Se distinguen 400 (archivo inválido), 413 (tamaño), 422 (entrada inválida o ausencia de intervenciones reconocidas), 401/403 (acceso o acción no permitida), 503 (dependencia ausente) y 502/504 (fallo de respuesta/timeout de Ollama). Los mensajes no reflejan cuerpos de solicitudes ni excepciones crudas.

4. Metodología del clasificador y restricciones clínicas

El código implementa una cascada: diccionario de firma profesional, reglas por términos del texto, predicción TF-IDF + Logistic Regression y fallback `otros`. El diccionario institucional pasó a un JSON privado configurado por entorno. Se conservaron valores, orden y comportamiento; el sistema devuelve 503 si falta el archivo, evitando resultados silenciosamente diferentes.

La segmentación identifica marcas temporales y tipos de bloque. El criterio existente exige un mínimo de 30 caracteres, descarta duplicados exactos normalizados y busca marcadores contables. Entre ellos figuran evolución clínica, nota de enfermería, seguimiento, interconsulta inicial, reevaluación y evaluaciones/actividades de distintas áreas.

La comparación por AST confirmó conservación de `REGLAS_CONTEO`, reglas de área e interconsultas y funciones seleccionadas de segmentación, clasificación y conteo. También se compararon todas las entradas del diccionario externalizado sin imprimir nombres. Se conservó el esquema de base.

Discrepancia pendiente	Impacto potencial	Propuesta de revalidación
README histórico enumeraba tres tipos; las reglas incluyen más	Diferencias en el denominador de intervenciones	Acordar especificación única y comparar contra bloques etiquetados
Estada inferida desde fechas del documento y exclusión de primeras fechas	Tasas por día distorsionadas	Contrastar episodios y fechas con fuente hospitalaria de referencia
Firmas y palabras clave dependientes de la institución	Sub/sobreasignación de áreas	Corpus independiente, revisión por profesionales y matriz por área
Duplicación exacta de texto y consolidación por archivo	Distinción imperfecta entre registros y pacientes únicos	Evaluar casos repetidos con identidad institucional controlada
UI histórica de confianza de IC frente a reglas `contar`	Interpretación clínica inconsistente	Acordar semántica de solicitud, respuesta y seguimiento

Estas discrepancias no fueron corregidas arbitrariamente. Cambiarlas exige decisión clínica, análisis del impacto sobre métricas y revalidación independiente.

La accuracy histórica de 92,98% sigue siendo metadata heredada del modelo: no fue medida durante este cierre y no se presenta como resultado de hoy. Las fixtures sintéticas comprueban ejecución y contratos, no desempeño poblacional, sesgos, utilidad clínica ni seguridad de recomendaciones.

5. Privacidad, pseudonimización y trazabilidad

El procesamiento se mantiene local. Los nuevos reportes NLP almacenan métricas y referencias pseudónimas, sin nombre original del archivo ni narrativa clínica. Se eliminó la devolución del original y del mapa identificatorio en los endpoints de pseudonimización. Los detalles de texto/evidencia clínica se suprimen de las salidas estructuradas.

`codigoHC` se revisó antes de cambiar su tratamiento: se usa para reconstruir Documento en PDF, deduplicar y conciliar HCD. Se mantiene durante el parseo; en persistencia/intercambio se transforma en una clave estable. Las pruebas comparan el número de grupos antes y después y verifican idempotencia. No se eliminó del parser ni se modificó el esquema.

Las cargas de censo usan temporales aislados, nombres únicos y reemplazo atómico al persistir. La UI ya no borra los uploads al cargarlos. Los protocolos RAG admitidos se identifican por SHA-256; no se escriben rutas arbitrarias ni se acepta una HC como documento desconocido del corpus. El índice revisado es independiente del índice histórico, que no se borró.

El logger estructurado registra timestamp, nivel, módulo, operación, request ID y estado HTTP. No registra body, query string ni texto de excepciones. Se deshabilitan access logs en los comandos recomendados. Las respuestas incluyen `Cache-Control: no-store`.

Límites que siguen vigentes

• Regex no detecta todas las entidades: nombres sin etiqueta, direcciones y cuasiidentificadores pueden escapar. Texto pseudonimizado no equivale a texto anónimo ni puede publicarse automáticamente.

• El salt público por defecto y los identificadores truncados heredados requieren migración planificada. Cambiar el salt durante un traslado rompe correspondencias; debe conservarse el valor institucional.

• Las bases y archivos históricos no se limpiaron ni borraron. Deben seguir bajo control institucional, con permisos y cifrado.

• La API ofrece Basic y restricción a loopback cuando no hay contraseña. No reemplaza roles, identidad institucional ni TLS. Streamlit necesita un perímetro de acceso propio antes de exponerse en red.

• Los nombres retirados del código y documentos retirados del árbol público siguen en commits anteriores. No se reescribió el historial ni se afirma eliminación retrospectiva.

El escaneo de publicación verifica rutas privadas, patrones de credenciales y coincidencias con el diccionario local sin imprimir valores. Es una revisión heurística complementaria; no prueba ausencia universal de PII. La identidad de la autora se incluye por instrucción expresa.

6. Fallos, causas raíz y correcciones

Prioridad / fallo	Causa raíz	Corrección y evidencia
P0: escape del corpus	Concatenación del filename con el directorio	Rechazo de rutas y allowlist por contenido; fallo original reproducido con fixture
P0: excepción identificable	`str(exc)` devuelto al cliente o HTML	Errores genéricos, sin inputs reflejados; regresiones HTTP
P0: original/mapa expuesto	Endpoint retornaba estructura interna completa	Respuesta reducida a texto transformado y cantidad
P0: narrativa/filename persistidos	Escritura directa de campos libres	Nuevos reportes estructurados, filename pseudónimo y texto extraído vacío
P0: nombres en código	Diccionario institucional embebido	Copia privada, exclusión Git, plantilla sintética y equivalencia
P1: recursos ausentes	Rutas relativas y carga sin control	Variables de entorno y 503 explícito para modelo/diccionario
P1: claves de censo expuestas	Exclusión parcial de columnas	Protección posterior al parseo; deduplicación verificada
P1: endpoints faltantes	Desfase README/API	Recuperación sin LLM y resumen local con validación
P1: RAG/LLM fallan sin contexto	Dependencias implícitas, respuestas sin validar	No descarga automática, índice vacío controlado y errores Ollama
P1: UI caída/vacíos	Requests sin control y acceso directo a claves	Timeouts, mensaje de API caída y tratamiento de reporte vacío
P1: Docker incompleto	Faltaban pipeline, puertos coherentes y persistencia	Imagen núcleo, healthchecks, volúmenes y validación real
P1: censos HTML/XLS válidos rechazados	Argumento `dtype` no soportado por `read_html`	Conversores por columna y pruebas de ceros iniciales; sin cambiar reglas
P1: versión del artifact	Dependencias sin fijar	scikit-learn 1.6.1, lock base y prueba de carga sin warning de versión

Las correcciones no borraron datos clínicos. Los siete archivos históricos retirados del índice Git permanecen en disco. Los cambios previos de la autora se conservaron y se documentaron por archivo, sin atribuir al hardening su lógica clínica o métricas preexistentes.

7. Validación ejecutada y evidencia

La validación final del directorio de trabajo obtuvo **68 aprobados, 0 fallidos y 0 omitidos**. Quedaron cinco advertencias: deprecación Starlette/httpx y cuatro advertencias del optimizador sobre opciones en el entrenamiento de modelos sintéticos. No se ocultaron ni se cambiaron expectativas para hacer pasar fallos.

Hubo una pasada intermedia con 62 aprobados y 1 fallo: “Resumen HCs” accedía a `resumen_ejecutivo` de una respuesta vacía. Se corrigió el código de UI; la siguiente pasada completa aprobó. La cobertura de formatos agregó dos fallos reproducidos en censo HTML/XLS por un argumento no admitido de pandas; se corrigió el lector y se agregaron cinco regresiones, incluyendo XLSX, HCD HTML y ceros iniciales. La suite final tiene 68 casos. Los fallos de instalación inicial y de TestClient dentro del sandbox se registraron como bloqueos de entorno, no como incompatibilidad de paquetes ni fallos clínicos.

Prueba	Resultado	Evidencia	Limitación
Baseline limpio	13/13	Auditoría técnica	Cobertura inicial limitada
Pytest final	68/68	pytest_resultados.xml	Fixtures sintéticas; algunos mocks
Health/docs/reportes/mobile	HTTP 200 real	validacion_http_real.json	Loopback, sin carga concurrente
Procesamiento y reinicio	Correctos	validacion_http_real.json	Modelo entrenado con fixture sintética
Artifact institucional	Carga/predicción sintética; 0 warnings de versión	evidencia_auditoria.json	No mide accuracy
Diccionario	Igualdad y comportamiento preservados	equivalencia_diccionario.json	Requiere archivo privado para repetir
Traversal y errores	Original reproducido; ataques rechazados tras corrección	evidencia_auditoria.json y pytest	Casos razonables, no pentest integral
Censo/privacidad/dedup	Aprobados	test_hardening.py	No corpus hospitalario real
Backup/restauración	Aprobados	test_backup.py	Tabla sintética, no backup real ejecutado
UI	13 módulos + 5 casos de API caída	test_frontend.py	API mock explícita
Streamlit servidor	Salud HTTP 200	validacion_http_real.json	No sesión clínica completa de navegador
Docker	Build/up/API/UI/proceso correctos	validacion_docker.json	Núcleo; privados sintéticos y volúmenes aislados
RAG API	Chroma/ST reales, subida y recuperación 200	validacion_rag_api.json	Dos documentos sintéticos
Ollama	Tags/generación 200	validacion_ollama.json	Salida no vacía, calidad no evaluada
Copia limpia	Instalación, pip check y 68 tests aprobados	validacion_instalacion_limpia.json	Plataforma actual; sin privados

La copia limpia se verificó sobre el commit técnico 1588703ce6da742842fe6e856918d716aa78f5f0: 68 aprobados, cero fallidos y cero omitidos, en 73,30 segundos. La última reconstrucción y ejecución Docker corresponde al mismo código: health API/UI y procesamiento sintético devolvieron HTTP 200.

Los tests de errores Ollama y separación recuperación/generación usan mocks y están nombrados como tales. La evaluación Chroma/SentenceTransformer, el servidor HTTP, Docker y la prueba breve de Ollama sí se ejecutaron con componentes reales, siempre sobre entradas sintéticas.

8. Evaluación reproducible de recuperación

Se versionó un corpus sintético de seis documentos técnicos y seis preguntas, cada una con un documento relevante. No se utilizaron registros de pacientes ni un LLM para asignar relevancia. El benchmark separa recuperación documental de generación y tiene dos variantes: referencia lexical TF-IDF y embeddings locales con ChromaDB real.

El modelo semántico usado fue `all-MiniLM-L6-v2`, revisión local `c9745ed1d9f207416be6d2e6f8de32d1f16199bf`. Se utilizó la caché ya presente, modo local/offline y un índice sintético independiente. No se descargaron modelos.

Variante / k	Recall@k	Precision@k	MRR@k
TF-IDF / 1	1,000	1,000	1,000
TF-IDF / 3	1,000	0,333	1,000
SentenceTransformer + Chroma / 1	1,000	1,000	1,000
SentenceTransformer + Chroma / 3	1,000	0,333	1,000

Recall@k es el número de relevantes recuperados dividido por relevantes etiquetados. Precision@k divide aciertos por k; con un único documento relevante, el máximo a k=3 en este fixture es 1/3. MRR@k promedia el inverso del primer rango relevante; es 0 cuando no aparece entre los k resultados. Las fórmulas se probaron también contra rankings conocidos.

Estos resultados perfectos de primer rango reflejan un benchmark pequeño y sencillo, con coincidencias temáticas claras. No miden recuperación clínica en el corpus institucional, preguntas ambiguas, negativos difíciles, cobertura, grounding, alucinaciones ni utilidad asistencial. La evaluación clínica del RAG sigue pendiente.

```
python -m rag.evaluation.evaluate --output /tmp/tfidf.json
python -m rag.evaluation.evaluate --backend sentence-transformer \
  --model-path /ruta/local/embeddings --output /tmp/semantica.json
```

Los JSON de entrega conservan rankings y relevancias para inspección. `/rag/consulta` no llama al LLM; `/rag/consultar` añade generación opcional. Ollama respondió una solicitud técnica breve con `tinyllama:latest` ya instalado. La calidad de esa respuesta no se puntuó ni se usó como evidencia de calidad clínica.

9. Instalación, inicio y configuración

Las versiones base están fijadas en `requirements-core.lock`, consumido por `requirements.txt`. Se verificó `pip check`. El lock completo Linux/Python 3.12 registra también el entorno RAG instalado; no debe trasladarse ciegamente a una plataforma diferente. El contenedor base excluye dependencias pesadas de embeddings y modelos privados. La imagen del sistema operativo no está fijada por digest: la reproducción no es byte a byte.

Instalación nueva

```
git clone --branch hardening-final \
  https://github.com/silvitagomez572-alt/sistema-hcd-ia.git
cd sistema-hcd-ia
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip check
cp .env.example .env
```

Configurar `.env` antes de procesar: base, artifact validado, diccionario institucional, uploads y salt. No sustituir el diccionario real por la plantilla sintética. No cargar pickles de origen no confiable. Nunca sobrescribir el entorno o configuración de una instalación existente sin respaldo.

Iniciar en esta máquina

```
cd /home/silvia/sistema-hcd-ia
source .venv-hardening/bin/activate
uvicorn api.main:app --host 127.0.0.1 --port 8000 --no-access-log
```

En otra terminal, mismo directorio y entorno:

```
streamlit run frontend/app.py --server.address=127.0.0.1 \
  --server.port=8501 --browser.gatherUsageStats=false
```

API: `http://127.0.0.1:8000/docs`; salud: `/health`; interfaz: `http://127.0.0.1:8501`. Si otro servicio ocupa esos puertos, revisar su configuración antes de detenerlo o elegir puertos libres y actualizar `HCD_API_URL`.

Docker y opcionales

```
# Configurar contraseña y permisos de los archivos privados según la guía.
docker compose config --quiet
docker compose build
docker compose up -d
```

El usuario del contenedor es UID 10001; otorgar lectura únicamente al proceso autorizado sobre diccionario/modelo. Los tests Docker montaron privados sintéticos y usaron puertos alternativos. RAG semántico en contenedor requiere dependencias e índice/modelo local adicionales; no está incluido en la imagen núcleo.

Para RAG local: instalar `requirements-rag.txt`, configurar `HCD_EMBEDDING_MODEL_PATH` y ejecutar `python -m pipeline.rag.indexar`. Para Ollama: `ollama serve` si no hay servicio activo, comprobar `curl http://localhost:11434/api/tags` y seleccionar un modelo ya instalado. No descargar modelos automáticamente.

10. Traslado a la computadora nueva

Solo se conoce 1 TB de almacenamiento. No se presupone RAM, GPU ni sistema operativo. La configuración del LLM queda pendiente del inventario real. El almacenamiento disponible no permite inferir capacidad de inferencia. La máquina actual pudo ejecutar las pruebas descritas; no se extrapola su rendimiento.

Backup y restauración

Detener nuevas cargas durante el respaldo del conjunto privado. Usar la API de backup SQLite para obtener una instantánea consistente; no copiar únicamente el `.db` mientras hay escrituras/WAL. El script verifica integridad y checksums, exige un destino inexistente y nunca sobrescribe el origen. No cifra: guardar y transferir el bundle en almacenamiento/canal institucional cifrado, fuera de Git.

```
python scripts/private_backup.py backup \
  --database /privado/hcs.db --destination /privado/backup_nuevo \
  --private /privado/profesionales.json \
  --private /privado/modelo_area_intervenciones.pkl \
  --private /privado/.env
python scripts/private_backup.py restore \
  --source /privado/backup_nuevo --destination /privado/restauracion_prueba
```

Los assets se guardan con nombres internos; el manifiesto privado conserva su correspondencia. Restaurar primero a una ubicación nueva, mapear rutas en `.env`, conservar el salt y verificar permisos. Copiar censos/uploads adicionales con una herramienta institucional, inventario de hashes y servicio detenido. Los volúmenes Docker no se transfieren al clonar Git. No usar `down -v` sobre datos reales. Mantener el origen como rollback.

Checklist de aceptación

1. Registrar SO, arquitectura, RAM, CPU, GPU/controladores y espacio libre; verificar cifrado y usuarios autorizados.

2. Instalar en entorno nuevo y ejecutar `pip check`; registrar diferencias de plataforma o resolución.

3. Verificar checksums, diccionario privado, artifact y SQLite restaurada, conservando el original.

4. Iniciar API/UI en loopback y comprobar health, docs y acceso autorizado.

5. Ejecutar pytest y `scripts/validate_local.py`: procesamiento sintético, reinicio y persistencia.

6. Verificar API caída/vacíos y consistencia UI/API; ensayar los formatos institucionales y OCR disponible.

7. Repetir benchmark RAG si hay embeddings; registrar versión del modelo/corpus e índice nuevo.

8. Seleccionar Ollama solo con hardware conocido y medir memoria, latencia y concurrencia con fixtures.

9. Validar restauración operativa, retención y rollback antes de introducir datos reales.

El procedimiento detallado se conserva en `docs/traslado_y_operacion.md` y no depende de compartir archivos privados por GitHub.

11. Limitaciones y requisitos hospitalarios pendientes

El núcleo es ejecutable y cuenta con evidencia reproducible de software en esta máquina. Permanecen pendientes la validación clínica independiente, la representatividad del clasificador, la evaluación del RAG sobre preguntas clínicas y una política de supervisión de generación. No se realizaron cambios clínicos para mejorar artificialmente métricas.

La trazabilidad actual registra eventos técnicos sin PII, pero no constituye auditoría institucional completa. Se requieren identidad y roles, TLS, auditoría durable, retención, cifrado, rotación de secretos y una revisión formal de reidentificación. La pseudonimización regex y el esquema de IDs heredado no equivalen a anonimización robusta.

La concurrencia SQLite tiene límites; no hubo prueba de carga, alta disponibilidad ni recuperación ante cortes eléctricos. La aplicación sigue teniendo monolitos y lógica de presentación que necesita refactor incremental posterior. Se conserva deuda en conciliación por paciente, fechas, interconsultas y métricas históricas. El pipeline de entrenamiento institucional y su corpus etiquetado no son reproducibles desde el repositorio público.

Para uso hospitalario real faltan: aprobación del servicio, ground truth revisado, evaluación por área y casos extremos, medición prospectiva, pruebas con formatos institucionales autorizados, operación de backups cifrados, soporte, responsabilidades y aceptación de seguridad. Las pruebas de hoy no sustituyen esos pasos.

Estado de Git y publicación

El código se registra en commits separados de funcionamiento/privacidad, pruebas y entorno/documentación, con la identidad Git de la autora. `entrega/PROVENIENCIA.md` distingue modificaciones previas. El PDF, fuente y evidencias se incorporan en un commit documental posterior.

Permanecen dos archivos previos sin commit, no incluidos en la publicación: `12.1.0` y `docs/auditoria_tecnica_integral_2026-07-07.md`. Los privados, entornos y documentos históricos excluidos se conservan localmente y no se publican. El informe no afirma publicación hasta verificar el push y la igualdad de hashes local/remoto.

La rama de entrega es exclusivamente `hardening-final`. No se hace push ni merge a `main`. El commit exacto publicado y el enlace directo al PDF se entregan junto con el archivo. Las evidencias y el código de validación permiten revisar qué se ejecutó realmente y qué sigue pendiente.